送信者グループを用いた公開鍵検索可能認証暗号の検索効率化について

吉田 岳史[1,a)]　江村 恵太[1,2,b)]

概要：公開鍵検索可能認証暗号 (PAEKS: Public Key Authenticated Encryption with Keyword Search (Huang, Li (Information Sciences 2017)) では, トラップドア (検索クエリ) からキーワードに関する情報が漏洩しないトラップドアプライバシーが定義されている. このトラップドアプライバシー達成のため, トラップドア生成時に送信者を指定している. キーワードに加えて送信者も一致する必要があるため, PAEKS の検索効率は送信者数に線形依存する. 本論文では, Wang ら (IEEE Transactions on Information Forensics and Security 2024) による送信者グループを用いた検索効率化アプローチ (グループ指向 PAEKS) を採用した, PAEKS の非対話鍵共有, 疑似ランダム置換, 公開鍵検索可能暗号からの一般的構成を提案する. 同じ送信者グループに属するグループメンバー間で同一の鍵ペアを共有することで, 検索効率を送信者グループ数への線形依存に改善する. なお Wang らの方式では, 同じ送信者グループに属するグループメンバー (内部者) に対し, トラップドアからキーワードの情報が必ず漏洩する (Yoshida-Emura, IEICE Transactions 2026). そこで本論文では, 外部者に対するトラップドア識別不可能性を定義, グループメンバーを除く攻撃者に対し, トラップドアからキーワードに関する情報が漏洩しないことを保証する. さらに内部者に対する暗号文-キーワード識別不可能性を定義, 同じ送信者グループに所属する送信者メンバに対しても暗号文からキーワードに関する情報が漏洩しないことを保証する. 提案一般的構成より, ペアリングや格子に基づく, グループ指向 PAEKS が構成可能である.

キーワード：公開鍵検索可能認証暗号, 検索効率改善, グループ指向

Search-Efficient Public-Key Authenticated Encryption with Keyword Search using Sender Groups

Takeshi Yoshida[1,a)]　Keita Emura[1,2,b)]

Abstract: Huang and Li (Information Sciences 2017) proposed public-key authenticated encryption with keyword search (PAEKS) and introduced trapdoor privacy, which guarantees that no keyword information is leaked from trapdoors. In PAEKS, a trapdoor must be generated for each sender when searching a keyword, resulting in search complexity that is typically linear in the number of senders. In this paper, we propose a generic construction of search-efficient PAEKS using sender groups, revisiting the group-oriented approach of Wang et al. (IEEE Transactions on Information Forensics and Security 2024). In our construction, users in the same sender group share a common key pair, shifting the search complexity from the number of senders to the number of sender groups. Motivated by the result of Yoshida and Emura (IEICE Transactions 2026), which showed that Wang et al.'s scheme fails to achieve trapdoor privacy against members of the same sender group, we define trapdoor indistinguishability against outsiders and ciphertext indistinguishability against insiders. The former protects keyword information in trapdoors from an adversary outside the group, while the latter protects keyword information in ciphertexts even from an adversary within the same sender group. Our generic construction provides pairing-based and lattice-based instantiations.

Keywords: Public Key Authenticated Encryption with Keyword Search, Search Complexity

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

公開鍵検索可能暗号 (PEKS: Public Key encryption with Keyword Search) [4] では, 暗号文を復号せずに, 暗号化されたキーワードが検索可能である. 送信者は受信者の公開鍵を指定してキーワード kw を暗号化, 受信者は自身の秘密鍵で検索キーワード kw' に紐づくトラップドアを生成する. テストアルゴリズムでは, 暗号文とトラップドアに対し, $kw = kw'$ であるかどうかを判定する. ここで PEKS では, 受信者の公開鍵を用いることで誰でも暗号文を生成可能であることに注意されたい. したがって, 攻撃者がトラップドアを入手した場合, 攻撃者は任意のキーワードを暗号化, トラップドアとともにテストアルゴリズムを実行することで, トラップドアに紐づくキーワードを推測可能である. 本攻撃は, 受信者の検索キーワードを推測可能な, キーワード推測攻撃 (KGA: Keyword Guessing Attacks) として知られている.

Huang ら [15] は, KGA 耐性を有する公開鍵検索可能認証暗号 (PAEKS: Public Key Authenticated Encryption with Keyword Search) を提案した. PAEKS では, キーワードの暗号化時に受信者の公開鍵に加え送信者の秘密鍵を用いることにより, 攻撃者による任意のキーワードの暗号化を防ぐことで, トラップドアプライバシーが定義可能となる. ここでトラップドア生成時に受信者の秘密鍵に加え, 送信者の公開鍵を用いる (トラップドア生成時に送信者個人を指定する) ことに注意されたい. すなわち, 受信者が単一のキーワードを検索する際でも送信者数だけトラップドアを生成する必要がある. 結果, 検索効率が送信者数に線形依存することとなる. PEKS では検索するキーワード数にのみ検索効率が依存していたことを鑑みると, KGA 耐性と引き換えに検索効率が悪化していると言える. KGA 耐性を持たせたまま, 如何にして検索効率を改善するかという課題に対して, 複数の対策手法が提案されている. 我々はこれらを, 以下の 3 つのアプローチに分類した.

グループ指向アプローチ [23]: 送信者グループを定義する. 受信者はトラップドア生成時, 送信者個人でなく送信者グループを指定する. 検索効率は送信者グループ数依存となる.

暗号文・トラップドア変換指向アプローチ [17, 18, 20, 24, 28]: PAEKS と同様, 送信者を指定する暗号文とトラップドアを生成する. それらを適切に変換し, 送信者に非依存な暗号文と送信者に非依存なトラップドアに更新する. 変換後の暗号文やトラップドアは PEKS と同様にキーワードにのみ依存するため, 検索効率が送信者数に依存しない.

サーバー指向アプローチ [9, 11, 25]: 送信者サーバーと受信者サーバーを定義する. 送信者は受信者サーバを指定して暗号文を生成, 送信者サーバーに送付する. 受信者は送信者サーバーを指定してトラップドアを生成, 受信者サーバーに送付する. 送信者サーバーは受け取った暗号文から, 送信者個人に関する情報を削除, 受信者サーバーは受け取ったトラップドアから, 受信者個人に関する情報を削除する. 結果, 変換後の暗号文とトラップドアは, 2 つのサーバー間の検索可能暗号となり, トラップドアは送信者数に非依存である.

検索効率改善の観点からすると, 暗号文・トラップドア変換やサーバー導入アプローチが望ましい. 一方で, プロキシサーバーや送受信者サーバーといった, 新規のエンティティを必要とする. 方式の複雑さが増すとともに, それぞれのエンティティを潜在的攻撃者とした安全性モデルの導入が必須となる. 暗号文・トラップドア変換のアプローチの中でも, プロキシを必要としない方式 [18] が提案されている. しかしテストサーバーがプロキシと同様の操作を実行することから, 依然としてテストサーバーに対しある程度の信頼が置かれている. これらの点を踏まえ, グループ指向アプローチに分類される, Wang らの ID ベースグループ検索可能暗号 (IBGEKS: Identity-Based Group Encryption With Keyword Search) は, PAEKS からの差分の小さい, 最も単純な方式であると言える.

ここで, Yoshida-Emura [27] により, トラップドアがキーワードをメッセージとした BLS (Boneh-Lynn-Shacham) 署名 [6] であることを利用した攻撃が提案されている. 送信者をグループに登録するための登録オラクル, またはトラップドアオラクルを通して BLS 署名の検証鍵に相当する値を取得, 検証式を実行することでトラップドアからキーワードに関する情報が漏洩していることを示した. さらに攻撃を一般化し, 正当性を満たす場合, 登録オラクルを利用可能なモデルで安全な IBGEKS 方式は実現不可能であることも示した. 本攻撃は PEKS におけるキーワード推測攻撃と類似しており, 登録オラクルを通じて入手した鍵を用いて任意のキーワードを暗号化している. 本脆弱性は, 安全性モデルを適切に定義することで回避可能である. しかし, IBGEKS の特定の脆弱性に対処したとしても, IBGEKS 方式はペアリングベースに限定されており, 耐量子性を有していない. 耐量子という観点からも, 様々な計算量仮定で具現化可能な一般的構成を提案することが望ましい.

本論文の貢献. 本論文では, グループ指向 PAEKS の一般的構成を提案する. 構成要素として, 非対話型鍵共有 (NIKE: Non-Interactive Key Exchange) [13], 疑似ランダ

[1] 金沢大学
Kanazawa University
[2] 産業技術総合研究所
National Institute of Advanced Industrial Science and Technology (AIST)
a) y0xxss1@stu.kanazawa-u.ac.jp
b) k-emura@se.kanazawa-u.ac.jp

© 2026 Information Processing Society of Japan

ム置換 (PRP: Pseudo-Random Permutation), PEKS を用いる．Abdalla ら [1] により, 匿名 ID ベース暗号 (IBE: Identity-Based Encryption) から PEKS が一般的に構成可能であることが示されている．(例: ペアリングベース IBE [5, 7, 22])．格子ベース IBE は, 基本的に匿名性を有する．NIKE に関して, ペアリングベース [13] や格子ベース [14] で構成可能である．以上から, 提案一般的構成より, ペアリングや格子に基づく, グループ指向 PAEKS が構成可能である．

グループ内の鍵共有について: Wang ら [23] が IBGEKS 方式で用いた, 送信者グループを用いた効率化アプローチを再検討した．我々の構成は, 送信者グループ内のメンバー間で鍵ペアを共有, 使用することで, 検索効率を送信者数依存から送信者グループ依存に改善する．

- 本構成では事前にグループ内で鍵共有がされたものと仮定する．(例: グループ鍵共有 [2, 8]) 詳細は 3 を参照のこと．
- IBGEKS [23] では, グループごとにグループマスター鍵 $\alpha \in \mathbb{Z}_p$ を定義し, 識別子 i の送信者メンバーはグループ秘密鍵 $H(\mathsf{ID}_i)^\alpha$ が与えられる．そして, トラップドア $H(kw)^\alpha$ も, 同様にグループマスター鍵を用いて生成されるため, グループ秘密鍵と同様の構造を持つ．構造の類似性を用いた IBGEKS の脆弱性が示されている [27]．本脆弱性を踏まえ, 我々の構成では, グループ内の鍵共有を用いた, より単純なアプローチを採用する．

Insider Full CI: グループ指向アプローチの場合, 同一の送信者グループに所属するメンバーに対しても, 暗号文からキーワードに関する情報が漏洩しない安全性モデルが望ましい．ただし Wang ら [23] はそのような状況を想定していない．そこで我々は, 同一の送信者グループに所属する攻撃者に対して, 暗号文からキーワードに関する情報が漏洩しないことを保証する, 内部者に対する完全暗号文-キーワード識別不可能性 (Insider full CI: full Ciphertext Indistinguishability against Insiders) を定義する．

Outsider TI: Yoshida-Emura [27] の結果により, IBGEKS 方式では, 同一グループに所属するメンバーに対し, トラップドアに紐づくキーワードに関する情報漏洩を防ぐことは不可能である．本指摘に基づき, 同一の送信者グループに属さない攻撃者に対して, トラップドアからキーワードに関する情報が漏洩しないことを保証する, 外部者に対するトラップドア-キーワード識別不可能性 (Outsider TI: Trapdoor Indistinguishability against Outsiders) を定義する．

提案一般的構成の概要: 提案一般的構成は, NIKE, PRP, PEKS から構成される．具体的には, 送信者-受信者間で NIKE 鍵を作成し, 送信者は自身の属するグループのメンバー全員と NIKE 鍵 $\mathsf{K}_{\mathsf{NIKE}}$ を共有する．送受信者は $\mathsf{K}_{\mathsf{NIKE}}$ を PRP F の鍵として, kw を疑似ランダムキーワード $\mathsf{pr}\text{-}kw \leftarrow F_{\mathsf{K}_{\mathsf{NIKE}}}(kw)$ に変換する．送信者は $\mathsf{pr}\text{-}kw$ をキーワードとする PEKS 暗号文を作成し, 受信者は $\mathsf{pr}\text{-}kw$ を検索キーワードとする。テスト時は, PEKS のテストアルゴリズムを用いる．

PEKS において, 送信者鍵は存在せず, 受信者の公開鍵を用いて誰でも暗号化可能でありながら, 暗号文からキーワードに関する情報は漏洩しない．言い換えれば, グループ内で共有された送信者グループの秘密鍵を攻撃者が入手し, 任意にキーワードの暗号化が可能でも, 暗号文に紐づくキーワードは秘匿される．PAEKS 攻撃者がチャレンジ送信者グループの秘密鍵として, NIKE 鍵を所持している場合, 攻撃者はチャレンジキーワードを自身で疑似ランダムキーワードに変換可能である．ただし, 疑似ランダムキーワードを PEKS チャレンジキーワードとみなすため, PEKS の安全性により, 暗号文からキーワードに関する情報は漏洩しない．

Insider full CI の攻撃者はトラップドアクエリ利用時, チャレンジキーワード (kw_0^*, kw_1^*) と異なる kw を送付する．この際, $F_{\mathsf{K}_{\mathsf{NIKE}}}(kw) = F_{\mathsf{K}_{\mathsf{NIKE}}}(kw_0^*)$ もしくは, $F_{\mathsf{K}_{\mathsf{NIKE}}}(kw) = F_{\mathsf{K}_{\mathsf{NIKE}}}(kw_1^*)$ を満たす場合, シミュレーションは失敗する．疑似ランダムキーワードの生成には, 疑似ランダム関数 (PRF: Pseudo-Random Function) を用いれば十分である．しかし内部攻撃者を仮定しているため, NIKE や PRF の安全性は利用できない．以上の理由から, 我々の構成では, PRF ではなく, 異なる入力に対し出力結果が重複しない PRP を用いる．より正確には, 2 つの異なる PRP 鍵 K, K' と, 2 つの異なるキーワード kw, kw' に対し, $F_K(kw) \neq F_{K'}(kw')$ を仮定する．

PEKS では, KGA によりトラップドアからキーワードに関する情報が漏洩する．そのため Outsider TI の攻撃者は, チャレンジトラップドアに紐づくチャレンジキーワード kw_b^* を入手可能であると仮定しても矛盾しない．外部攻撃者はチャレンジ送信者グループの秘密鍵の取得が許されず, すなわち NIKE 鍵= PRP 鍵も知らない．そのため PRP の疑似ランダム性により, b が秘匿される．ただし, 我々の構成は Outsider full TI 安全性を満たさない．ここで full 安全性は, 攻撃者がチャレンジキーワードをトラップドアクエリに送付可能であることを意味する．full 安全ではないことは, PEKS においてトラップドア生成アルゴリズムが決定的である場合が存在することよりわかる (例: Abdalla ら [5] の Boneh-Franklin IBE からの PEKS の構成)．Insider full CI かつ Outsider full TI を満たす検索効率型グループ指向 PAEKS の構成は, 今後の課題とする．

既存一般的構成との関係性: 我々の提案一般的構成は, どち

© 2026 Information Processing Society of Japan

らも PAEKS の一般的構成である, Li と Boyen [19] の手法と, Emura [12] の手法のハイブリッドとみなすことができる. Li-Boyen の構成では, 送受信者間で確立した NIKE 鍵を, Symmetric-key Equality-Predicate Encryption (EPE) の共通鍵として用いている. ここで NIKE の対称性により, 送信者であっても受信者の公開鍵と自身の秘密鍵を用いることでトラップドアを作成可能であることに注意されたい. すなわち, Yoshida と Emura [26] が指摘しているように, Li-Boyen 構成は内部者に対する CI 安全性を満たさない. ただし CI 安全性のモデル外攻撃であり, Li-Boyen 構成の不備を指摘するものではないことに注意されたい.

Emura の構成では, Smooth projective hash functions (SPHFs) を導入し, SPHF-based password-authenticated key exchange (PAKE) プロトコル [3,16] を通じて, 送受信者間で鍵共有を行う. 共有された PAKE 鍵は, PRF の鍵として用い, キーワード kw を疑似ランダムキーワード pr-kw に変換する. 送信者は pr-kw を PEKS の暗号化アルゴリズムで暗号化し, 受信者は pr-kw を検索キーワードとしてトラップドアを作成する. テストアルゴリズムは構成要素の PEKS のテストアルゴリズムを用いる. SPHF に起因して, 送信者公開鍵は受信者公開鍵を用いて生成する必要がある. すなわち, 受信者指定型 PAEKS の一般的構成と言える.

我々の提案一般的構成では, 受信者指定を避けるため, Emura の構成における鍵共有プロセスを SPHF-based PAKE から NIKE に置き換える. さらに Emura の構成における PRF を PRP に, Li-Boyen 構成における EPE を PEKS に置き換えることで, insider full CI 安全性を担保する. 以上から, 我々の一般的構成は, 両一般的構成の強みにより他方式の制限を補った構成とみなせる.

2. 準備

本章では, 提案一般的構成の構成要素を定義する. ページ数の都合によりシンタックスのみ記載, 各安全性定義は付録を参照されたい.

疑似ランダム置換 (PRP): λ をセキュリティパラメータ, ℓ を多項式関数とする. 疑似ランダム置換を $\mathsf{PRP} = \{F_k : \{0,1\}^{\ell(\lambda)} \to \{0,1\}^{\ell(\lambda)}\}$ と定義する.

提案一般的構成では, PRP を用いてキーワードを疑似ランダムなキーワード pr-$kw \leftarrow F_{\mathsf{K_{NIKE}}}(kw)$ に変換する. ここで, 2 つの異なる PRP 鍵 K, K' と, 2 つの異なるキーワード kw, kw' に対し, $F_K(kw) = F_{K'}(kw')$ となる事態が発生しないことを保証する必要がある. これは, ハッシュ関数 $H : \{0,1\}^{|K|+|kw|} \to \{0,1\}^{|kw|}$, $F_K(kw) := H(K||kw)$ を導入しランダム関数とみなすことで, その衝突困難性によって回避可能である.

非対話鍵共有 (NIKE): 以下, 非対話鍵共有方式 $\mathsf{NIKE} = (\mathsf{NIKE.SetUp}, \mathsf{NIKE.Gen}, \mathsf{NIKE.SharedKey})$ を定義する.

Definition1 (シンタックス)

$\mathsf{NIKE.SetUp}(1^\lambda)$**:** セットアップアルゴリズムはセキュリティパラメータ $\lambda \in \mathbb{N}$ を入力とし, 公開パラメータ $\mathsf{pp_{NIKE}}$ を出力する.

$\mathsf{NIKE.Gen}(\mathsf{NIKE.ID})$**:** 鍵生成アルゴリズムは識別子 NIKE.ID を入力とし, 公開鍵 NIKE.pk と, 秘密鍵 NIKE.sk を出力する.

$\mathsf{NIKE.SharedKey}(\mathsf{NIKE.ID}_1, \mathsf{NIKE.pk}_1, \mathsf{NIKE.ID}_2, \mathsf{NIKE.sk}_2)$**:** 鍵共有アルゴリズムは 2 つの識別子 $\mathsf{NIKE.ID}_1, \mathsf{NIKE.ID}_2$ と, 識別子に対応する鍵 $\mathsf{NIKE.pk}_1, \mathsf{NIKE.sk}_2$ を入力とし, NIKE 鍵 $\mathsf{K_{NIKE}}$ もしくは $\perp$ を出力する.

公開鍵検索可能暗号 (PEKS): 以下, 公開鍵検索可能暗号方式 $\mathsf{PEKS} = (\mathsf{PEKS.SetUp}, \mathsf{PEKS.KGen}, \mathsf{PEKS.Enc}, \mathsf{PEKS.Trapdoor}, \mathsf{PEKS.Test})$ を定義する.

Definition2 (シンタックス)

$\mathsf{PEKS.SetUp}(1^\lambda)$**:** セットアップアルゴリズムはセキュリティパラメータ $\lambda \in \mathbb{N}$ を入力とし, 公開パラメータ $\mathsf{pp_{PEKS}}$ を出力する.

$\mathsf{PEKS.KGen}(\mathsf{pp_{PEKS}})$**:** 鍵生成アルゴリズムは $\mathsf{pp_{PEKS}}$ を入力とし, 公開鍵 $\mathsf{pk_{PEKS}}$ と秘密鍵 $\mathsf{sk_{PEKS}}$ を出力する.

$\mathsf{PEKS.Enc}(\mathsf{pp_{PEKS}}, \mathsf{pk_{PEKS}}, kw)$**:** 暗号化アルゴリズムは $\mathsf{pp_{PEKS}}, \mathsf{pk_{PEKS}}$ とキーワード kw を入力とし, $\mathsf{C_{PEKS},}_{kw}$ を出力する.

$\mathsf{PEKS.Trapdoor}(\mathsf{pp_{PEKS}}, \mathsf{sk_{PEKS}}, kw')$**:** トラップドア生成アルゴリズムは $\mathsf{pp_{PEKS}}, \mathsf{sk_{PEKS}}$ と検索キーワード kw' を入力とし $\mathsf{T_{PEKS},}_{kw'}$.

$\mathsf{PEKS.Test}(\mathsf{C_{PEKS},}_{kw}, \mathsf{T_{PEKS},}_{kw'})$**:** テストアルゴリズムは $\mathsf{C_{PEKS},}_{kw}, \mathsf{T_{PEKS},}_{kw'}$ を入力とし, 1 もしくは 0 を出力する.

3. グループ指向 PAEKS

PAEKS では, 暗号化時に送信者の秘密鍵を, トラップドア生成時には送信者の公開鍵を用いることで送信者を指定する. 標準的な PAEKS では送信者鍵は送信者個人に紐づくが, グループ指向 PAEKS では, 送信者鍵を送信者グループのメンバー全員に共有すると仮定することで, 送信者鍵を送信者グループに紐づける. ただし, 送信者グループメンバーへの鍵共有は定式化せず, グループ鍵生成アルゴリズムの実行と同時に既に行われたものと暗に仮定する. よって鍵共有の手法は任意に設定可能であり, 最も単純な手法だとグループ代表者がグループ鍵生成を実行し, グループメンバーに分配するだけで成立する. IBGEKS [23] の登録アルゴリズムと同様に, ID ベース設定に変更し, 鍵生成センタ (KGC: Key Generation Center) を通じてグループメンバーに鍵を発行することも可能である. これらの手法ではグループ代表者や KGC に信頼を置く必要があるが, グループ鍵共有 (GKA: Group Key Agreement) の導入に

© 2026 Information Processing Society of Japan

より信頼のおける第 3 者は不要となる (例: 離散対数ベース [8], 格子ベース [2]). 具体的には, グループメンバーに対し乱数と識別不可能な鍵 k を共有し, k をグループ鍵生成アルゴリズム $(\mathsf{pk_{SG}}, \mathsf{sk_{SG}}) \leftarrow \mathsf{KGen_{SG}}(\mathsf{pp}, \mathsf{ID_{SG}}; k)$ 実行時の乱数として扱えばよい. 当然グループメンバーとの鍵共有を省略することで, 標準的な PAEKS と同様に, 送信者鍵を送信者個人に紐づけることも可能である. 以後, 送信者グループ内の鍵共有プロセスは明示的に記載しない.

以下, グループ指向公開鍵検索可能認証暗号 $\mathsf{PAEKS} = (\mathsf{SetUp}, \mathsf{KGen_{SG}}, \mathsf{KGen_R}, \mathsf{Enc}, \mathsf{Trapdoor}, \mathsf{Test})$ を定義する.

Definition3 (シンタックス)

$\mathsf{SetUp}(1^\lambda)$**:** セットアップアルゴリズムはセキュリティパラメータ $\lambda \in \mathbb{N}$ を入力とし, 公開パラメータ pp を出力する.

$\mathsf{KGen_{SG}}(\mathsf{pp}, \mathsf{ID_{SG}})$**:** 送信者グループ鍵生成アルゴリズムは, pp と送信者グループの識別子 $\mathsf{ID_{SG}}$ を入力とし, 送信者グループの公開鍵 $\mathsf{pk_{SG}}$ と秘密鍵 $\mathsf{sk_{SG}}$ を出力する. ただし, $\mathsf{ID_{SG}}$ は $\mathsf{pk_{SG}}$ に埋め込まれており, $\mathsf{pk_{SG}}$ から $\mathsf{ID_{SG}}$ を識別可能である.

$\mathsf{KGen_R}(\mathsf{pp}, \mathsf{ID_R})$**:** 受信者鍵生成アルゴリズムは, pp と受信者の識別子 $\mathsf{ID_R}$ を入力とし, 受信者の公開鍵 $\mathsf{pk_R}$ と秘密鍵 $\mathsf{sk_R}$. ただし, $\mathsf{ID_R}$ は $\mathsf{pk_R}$ に埋め込まれており, $\mathsf{pk_R}$ から $\mathsf{ID_R}$ を識別可能である.

$\mathsf{Enc}(\mathsf{pp}, \mathsf{pk_{SG}}, \mathsf{pk_R}, \mathsf{sk_{SG}}, kw)$**:** 暗号化アルゴリズムは $\mathsf{pp}, \mathsf{pk_R}, \mathsf{pk_{SG}}, \mathsf{sk_{SG}}$ とキーワード kw を入力とし, 暗号文 C_{kw} を出力する.

$\mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk_R}, \mathsf{pk_{SG}}, \mathsf{sk_R}, kw')$**:** トラップドア生成アルゴリズムは, $\mathsf{pp}, \mathsf{pk_R}, \mathsf{pk_{SG}}, \mathsf{sk_R}$ と検索キーワード kw' を入力とし, トラップドア $\mathsf{T}_{kw'}$ を出力する.

$\mathsf{Test}(\mathsf{C}_{kw}, \mathsf{T}_{kw'})$**:** テストアルゴリズムは, $\mathsf{C}_{kw}, \mathsf{T}_{kw'}$ を入力とし, 1 もしくは 0 を出力する.

Definition4 (正当性) 全ての $\lambda \in \mathbb{N}, \mathsf{pp} \leftarrow \mathsf{SetUp}(1^\lambda)$, $\mathsf{ID_{SG}}, \mathsf{ID_R}, (\mathsf{pk_{SG}}, \mathsf{sk_{SG}}) \leftarrow \mathsf{KGen_{SG}}(\mathsf{pp}, \mathsf{ID_{SG}}), (\mathsf{pk_R}, \mathsf{sk_R}) \leftarrow \mathsf{KGen_R}(\mathsf{pp}, \mathsf{ID_R})$, $kw \in \mathcal{KW}$ に対し, $1 \leftarrow \mathsf{Test}(\mathsf{Enc}(\mathsf{pp}, \mathsf{pk_R}, \mathsf{pk_{SG}}, \mathsf{sk_{SG}}, kw), \mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk_R}, \mathsf{pk_{SG}}, \mathsf{sk_R}, kw))$ が常に出力される場合, PAEKS は正当性を満たすと定義する.

PAEKS の一貫性を以下に定義する, これは, 暗号化時に指定した送受信者と用いたキーワードに対し, トラップドア生成時に指定した送受信者と用いた検索キーワードが全て一致している場合, テストアルゴリズムが常に 0 を出力することを保証する.

Definition5 (一貫性) $\mathcal{C}$ は $\mathsf{pp} \leftarrow \mathsf{SetUp}(1^\lambda)$, $(\mathsf{pk_{SG_0}}, \mathsf{sk_{SG_0}}) \leftarrow \mathsf{KGen_{SG}}(\mathsf{pp}, \mathsf{ID_{SG_0}}), (\mathsf{pk_{SG_1}}, \mathsf{sk_{SG_1}}) \leftarrow \mathsf{KGen_{SG}}(\mathsf{pp}, \mathsf{ID_{SG_1}}), (\mathsf{pk_{R_0}}, \mathsf{sk_{R_0}}) \leftarrow \mathsf{KGen_R}(\mathsf{pp}, \mathsf{ID_{R_0}}), (\mathsf{pk_{R_1}}, \mathsf{sk_{R_1}}) \leftarrow \mathsf{KGen_R}(\mathsf{pp}, \mathsf{ID_{R_1}})$ を実行し, $\mathsf{pp}, \mathsf{pk_{SG_0}}, \mathsf{pk_{SG_1}}, \mathsf{pk_{R_0}}, \mathsf{pk_{R_1}}$ を $\mathcal{A}$ に送付する. $\mathcal{A}$ は $(i, j, kw, i', j', kw') \in (\{0,1\}^2 \times \mathcal{KW})^2$ を $\mathcal{C}$ に出力する. $\mathcal{C}$ が $\mathsf{C}_{kw} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk_{SG_i}}, \mathsf{pk_{R_j}}, \mathsf{sk_{SG_i}}, kw)$ と $\mathsf{T}_{kw'} \leftarrow \mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk_{R_{j'}}}, \mathsf{pk_{SG_{i'}}}, \mathsf{sk_{R_{j'}}}, kw')$ を実行し. $(i, j, kw) \neq (i', j', kw')$ かつ $1 \leftarrow \mathsf{Test}(\mathsf{C}_{kw}, \mathsf{T}_{kw'})$ を満たす場合に $\mathcal{A}$ は勝利すると定義する. 全ての PPT 攻撃者に対し, 利得 $\mathsf{Adv}^{\mathsf{consist}}_{\mathsf{PAEKS},\mathcal{A}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$ が λ に対し無視できる場合, PAEKS は一貫性を満たすと定義する.

内部者に対する完全暗号文-キーワード識別不可能性 (Insider full CI) を定義する. これは同じ送信者グループに所属する攻撃者に対して, 暗号文からキーワードに関する情報が漏洩しないことを保証する, ここで, full 安全性とは攻撃者がチャレンジキーワードの暗号文を入手可能であることを意味している. Li-Boyen [19] の full CI の定義では, $\mathcal{A}$ はトラップドアオラクルに $(\mathsf{pk_S}, kw)$ を送付している. このクエリは一見して, 攻撃者が不正に作成した公開鍵を受理可能な定義であり, その場合, NIKE の不正な鍵登録を許可する DKR-CKS 安全性が必要だと考えられる. しかし, 安全性証明では, NIKE のユーザー鍵が HKR-CKS のチャレンジャーによって生成されていない場合, シミュレーター自身が鍵生成 $(\mathsf{pk_S}, \mathsf{sk_S}) \leftarrow \mathsf{NIKE.Gen}(\mathsf{S})$ を実行した上で, $\mathsf{pk_S}$ をトラップドア生成アルゴリズムに入力している. 提案一般的構成では, より状況を厳密に定義するため, クエリに送信する $\mathsf{pk_{SG}}$ を $\mathsf{ID_{SG}}$ に置き換える. また, $\mathcal{A}$ が全ての送信者グループの秘密鍵を所持可能なモデルであり, 任意のキーワードおよび送信者グループの秘密鍵で暗号化可能であるため, 暗号化オラクルに関してもここでは定義しない. また, 暗号化時に任意の受信者を指定可能であることから, 受信者鍵生成オラクルも定義しない.

Definition6 (Insider Full CI)

セットアップ: $\mathcal{C}$ は $\mathsf{pp} \leftarrow \mathsf{SetUp}(1^\lambda), (\mathsf{pk_{R^*}}, \mathsf{sk_{R^*}}) \leftarrow \mathsf{KGen_R}(\mathsf{pp}, \mathsf{ID_{R^*}})$ を実行し, $\mathsf{pp}, \mathsf{pk_{R^*}}$ を $\mathcal{A}$ に送付する. $\mathcal{C}$ はオラクルリスト $\mathcal{L}_{\mathsf{KGCorr}} \leftarrow \emptyset$ と $\mathcal{L}_{\mathsf{Trapdoor}} \leftarrow \emptyset$ を定義する.

Phase I: $\mathcal{A}$ は以下のオラクルを利用可能.

送信者グループ鍵生成およびコラプトオラクル: $\mathcal{A}$ は $(\mathsf{ID_{SG}}, \cdot, \cdot) \notin \mathcal{L}_{\mathsf{KGCorr}}$ を満たす $\mathsf{ID_{SG}}$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{pk_{SG}}, \mathsf{sk_{SG}}) \leftarrow \mathsf{KGen_{SG}}(\mathsf{pp}, \mathsf{ID_{SG}})$ を $\mathcal{A}$ に返し, 記録 $\mathcal{L}_{\mathsf{KGCorr}} \leftarrow \mathcal{L}_{\mathsf{KGCorr}} \cup \{(\mathsf{ID_{SG}}, \mathsf{pk_{SG}}, \mathsf{sk_{SG}})\}$ を行う.

トラップドアオラクル: $\mathcal{A}$ は $(\mathsf{ID_{SG}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KGCorr}}$ を満たす $(\mathsf{ID_{SG}}, kw)$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $\mathcal{L}_{\mathsf{KGCorr}}$ から $(\mathsf{ID_{SG}}, \mathsf{pk_{SG}}, \mathsf{sk_{SG}})$ を参照, $\mathsf{T}_{kw} \leftarrow \mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk_{R^*}}, \mathsf{pk_{SG}}, \mathsf{sk_{R^*}}, kw)$ を返し, 記録 $\mathcal{L}_{\mathsf{Trapdoor}} \leftarrow \mathcal{L}_{\mathsf{Trapdoor}} \cup \{(\mathsf{ID_{SG}}, kw)\}$ を行う.

チャレンジ: $\mathcal{A}$ は $(\mathsf{ID_{SG^*}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KGCorr}}$, $(\mathsf{ID_{SG^*}}, kw_0^*) \notin \mathcal{L}_{\mathsf{Trapdoor}}, (\mathsf{ID_{SG^*}}, kw_1^*) \notin \mathcal{L}_{\mathsf{Trapdoor}}$ を満たす $(\mathsf{ID_{SG^*}}, kw_0^*, kw_1^*)$ を宣言する. $\mathcal{C}$ は $b \xleftarrow{\$} \{0,1\}$ を選択し, チャレンジ暗号文 $\mathsf{C}_{kw_b^*} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk_{R^*}}, \mathsf{pk_{SG^*}}, \mathsf{sk_{SG^*}}, kw_b^*)$ を $\mathcal{A}$ に送

© 2026 Information Processing Society of Japan

付する.

Phase II: $\mathcal{A}$ は以下のオラクルを利用可能. .

送信者グループ鍵生成およびコラプトオラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \notin \mathcal{L}_{\mathsf{KGCorr}}$ を満たす $\mathsf{ID}_{\mathsf{SG}}$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \leftarrow \mathsf{KGen}_{\mathsf{SG}}(\mathsf{pp}, \mathsf{ID}_{\mathsf{SG}})$ を $\mathcal{A}$ に返し, 記録 $\mathcal{L}_{\mathsf{KGCorr}} \leftarrow \mathcal{L}_{\mathsf{KGCorr}} \cup \{(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}})\}$ を行う.

トラップドアオラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KGCorr}}$, $(\mathsf{ID}_{\mathsf{SG}}, kw) \neq (\mathsf{ID}_{\mathsf{SG}^*}, kw_0^*)$, $(\mathsf{ID}_{\mathsf{SG}}, kw) \neq (\mathsf{ID}_{\mathsf{SG}^*}, kw_1^*)$ を満たす $(\mathsf{ID}_{\mathsf{SG}}, kw)$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $\mathcal{L}_{\mathsf{KGCorr}}$ から $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}})$ を参照, $\mathsf{T}_{kw} \leftarrow \mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk}_{\mathsf{R}^*}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{R}^*}, kw)$ を返し, 記録 $\mathcal{L}_{\mathsf{Trapdoor}} \leftarrow \mathcal{L}_{\mathsf{Trapdoor}} \cup \{(\mathsf{ID}_{\mathsf{SG}}, kw)\}$ を行う.

推測: $\mathcal{A}$ は $b' \in \{0,1\}$ を出力する. 全ての PPT 攻撃者 $\mathcal{A}$ に対し, 利得 $\mathsf{Adv}^{\mathsf{InsiderFullCI}}_{\mathsf{PAEKS},\mathcal{A}}(\lambda) = |\Pr[b = b'] - 1/2|$ が λ に対し無視できる場合, PAEKS は Insider full CI を満たすと定義する.

外部者に対するトラップドア-キーワード識別不可能性 (Outsider TI) を定義する. これは同一の送信者グループに属さない攻撃者に対して, トラップドアからキーワードに関する情報が漏洩しないことを保証する. 提案一般的構成において, TI に関しては full を満たさず, 同一のキーワード, ユーザー間でトラップドアを生成したという情報の漏洩を許容する. CI 安全性においては, full を満たすことが標準として要求されているが, TI 安全性に対しては比較的寛容である. というのも, full TI の達成には相応のコストを要する. Li-Boyen の一般的構成でも, ペアリングフリーな離散対数ベースの方式で, full TI を満たす PAEKS は得られていない. さらに, システムモデルとしては, 暗号文とは異なり, トラップドアは永久的に保存するものではなく, 一時的なクエリとして用いられる. これらの理由から, full ではない TI 安全性であっても, 安全性として有意義なものであると言える. Outsider TI において, $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{ID}_{\mathsf{R}}, kw)$ を $\mathcal{C}$ に送付し, 自身で $\mathsf{Enc}(\mathsf{pp}, \mathsf{pk}_{\mathsf{SG}^*}, \mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{SG}^*}, kw)$ を実行することはできない. ここで, $(\mathsf{ID}_{\mathsf{R}}, kw)$ を送付し $\mathsf{Enc}(\mathsf{pp}, \mathsf{pk}_{\mathsf{SG}^*}, \mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{SG}^*}, kw)$ を得るだけでも十分に見える. しかし, この $\mathsf{ID}_{\mathsf{SG}^*}$ はコラプトされていないものに限定するため, 暗号化オラクルでは $\mathsf{ID}_{\mathsf{SG}}$ を指定する必要がある.

Definition7 (Outsider TI)

セットアップ: $\mathcal{C}$ は $\mathsf{pp} \leftarrow \mathsf{SetUp}(1^\lambda), (\mathsf{pk}_{\mathsf{R}^*}, \mathsf{sk}_{\mathsf{R}^*}) \leftarrow \mathsf{KGen}_{\mathsf{R}}(\mathsf{pp}, \mathsf{ID}_{\mathsf{R}^*})$ を実行し, $\mathsf{pp}, \mathsf{pk}_{\mathsf{R}^*}$ を $\mathcal{A}$ に送付する. $\mathcal{C}$ はオラクルリスト $\mathcal{L}_{\mathsf{KeyGenSG}} \leftarrow \emptyset$, $\mathcal{L}_{\mathsf{CorruptSG}} \leftarrow \emptyset$, $\mathcal{L}_{\mathsf{KeyGenR}} \leftarrow \emptyset$, $\mathcal{L}_{\mathsf{Enc}} \leftarrow \emptyset$, $\mathcal{L}_{\mathsf{Trapdoor}} \leftarrow \emptyset$ を定義する.

Phase I: $\mathcal{A}$ は以下のオラクルを利用可能.

送信者グループ鍵生成オラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \notin \mathcal{L}_{\mathsf{KeyGenSG}}$ を満たす $\mathsf{ID}_{\mathsf{SG}}$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \leftarrow \mathsf{KGen}_{\mathsf{SG}}(\mathsf{pp}, \mathsf{ID}_{\mathsf{SG}})$ を実行, $\mathsf{pk}_{\mathsf{SG}}$ を $\mathcal{A}$ に返し, 記録 $\mathcal{L}_{\mathsf{KeyGenSG}} \leftarrow \mathcal{L}_{\mathsf{KeyGenSG}} \cup \{(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}})\}$ を行う.

コラプトオラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenSG}}$ を満たす $\mathsf{ID}_{\mathsf{SG}}$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \in \mathcal{L}_{\mathsf{KeyGenSG}}$ を参照, $\mathsf{sk}_{\mathsf{SG}}$ を $\mathcal{A}$ に返し. 記録 $\mathcal{L}_{\mathsf{CorruptSG}} \leftarrow \mathcal{L}_{\mathsf{CorruptSG}} \cup \{\mathsf{ID}_{\mathsf{SG}}\}$ を行う.

受信者鍵生成オラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{R}}, \cdot, \cdot) \notin \mathcal{L}_{\mathsf{KeyGenR}}$ を満たす ID_{R} を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{R}}) \leftarrow \mathsf{KGen}_{\mathsf{R}}(\mathsf{pp}, \mathsf{ID}_{\mathsf{R}})$ を実行, pk_{R} を返し, 記録 $\mathcal{L}_{\mathsf{KeyGenR}} \leftarrow \mathcal{L}_{\mathsf{KeyGenR}} \cup \{(\mathsf{ID}_{\mathsf{R}}, \mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{R}})\}$ を行う.

暗号化オラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenSG}}$, $(\mathsf{ID}_{\mathsf{R}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenR}}$ を満たす $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{ID}_{\mathsf{R}}, kw)$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \in \mathcal{L}_{\mathsf{KeyGenSG}}$ と $(\mathsf{ID}_{\mathsf{R}}, \mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{R}}) \in \mathcal{L}_{\mathsf{KeyGenR}}$ を参照し, $\mathsf{C}_{kw} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{SG}}, kw)$ を実行する. $\mathcal{C}$ は $\mathsf{C}_k w$ を返し, 記録 $\mathcal{L}_{\mathsf{Enc}} \leftarrow \mathcal{L}_{\mathsf{Enc}} \cup \{(\mathsf{ID}_{\mathsf{SG}}, \mathsf{ID}_{\mathsf{R}}, kw)\}$ を行う.

トラップドアオラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenSG}}$ を満たす $(\mathsf{ID}_{\mathsf{SG}}, kw)$ を $\mathcal{A}$ に送付する. $\mathcal{C}$ は $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \in \mathcal{L}_{\mathsf{KeyGenSG}}$ を参照, $\mathsf{T}_{kw} \leftarrow \mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk}_{\mathsf{R}^*}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{R}^*}, kw)$ を返し, 記録 $\mathcal{L}_{\mathsf{Trapdoor}} \leftarrow \mathcal{L}_{\mathsf{Trapdoor}} \cup \{(\mathsf{ID}_{\mathsf{SG}}, kw)\}$ を行う.

チャレンジ: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}^*}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenSG}}, \mathsf{ID}_{\mathsf{SG}^*} \notin \mathcal{L}_{\mathsf{CorruptSG}}, (\mathsf{ID}_{\mathsf{SG}^*}, \mathsf{ID}_{\mathsf{R}^*} kw_0^*) \notin \mathcal{L}_{\mathsf{Enc}}, (\mathsf{ID}_{\mathsf{SG}^*}, \mathsf{ID}_{\mathsf{R}^*}, kw_1^*,) \notin \mathcal{L}_{\mathsf{Enc}}$, $(\mathsf{ID}_{\mathsf{SG}^*}, kw_0^*) \notin \mathcal{L}_{\mathsf{Trapdoor}}$, $((\mathsf{ID}_{\mathsf{SG}^*}, kw_1^*) \notin \mathcal{L}_{\mathsf{Trapdoor}}$ を満たす $(\mathsf{ID}_{\mathsf{SG}^*}, kw_0^*, kw_1^*)$ を宣言する. $\mathcal{C}$ は $b \xleftarrow{\$} \{0,1\}$ を選択し, チャレンジトラップドア $\mathsf{T}_{kw_b^*} \leftarrow \mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk}_{\mathsf{R}^*}, \mathsf{pk}_{\mathsf{SG}^*}, \mathsf{sk}_{\mathsf{R}^*}, kw_b^*)$ を $\mathcal{A}$ に送付する.

Phase II: $\mathcal{A}$ は以下のオラクルを利用可能.

送信者グループ鍵生成オラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \notin \mathcal{L}_{\mathsf{KeyGenSG}}$ を満たす $\mathsf{ID}_{\mathsf{SG}}$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \leftarrow \mathsf{KGen}_{\mathsf{SG}}(\mathsf{pp}, \mathsf{ID}_{\mathsf{SG}})$ を実行, $\mathsf{pk}_{\mathsf{SG}}$ を $\mathcal{A}$ に返し, 記録 $\mathcal{L}_{\mathsf{KeyGenSG}} \leftarrow \mathcal{L}_{\mathsf{KeyGenSG}} \cup \{(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}})\}$ を行う.

コラプトオラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenSG}}$, $\mathsf{ID}_{\mathsf{SG}} \neq \mathsf{ID}_{\mathsf{SG}^*}$ を満たす $\mathsf{ID}_{\mathsf{SG}}$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \in \mathcal{L}_{\mathsf{KeyGenSG}}$ を参照, $\mathsf{sk}_{\mathsf{SG}}$ を $\mathcal{A}$ に返し. 記録 $\mathcal{L}_{\mathsf{CorruptSG}} \leftarrow \mathcal{L}_{\mathsf{CorruptSG}} \cup \{\mathsf{ID}_{\mathsf{SG}}\}$ を行う.

受信者鍵生成オラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{R}}, \cdot, \cdot) \notin \mathcal{L}_{\mathsf{KeyGenR}}$ を満たす ID_{R} を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{R}}) \leftarrow \mathsf{KGen}_{\mathsf{R}}(\mathsf{pp}, \mathsf{ID}_{\mathsf{R}})$ を実行, pk_{R} を返し, 記録 $\mathcal{L}_{\mathsf{KeyGenR}} \leftarrow \mathcal{L}_{\mathsf{KeyGenR}} \cup \{(\mathsf{ID}_{\mathsf{R}}, \mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{R}})\}$ を行う.

暗号化オラクル: $\mathcal{A}$ は $(\mathsf{ID}_{\mathsf{SG}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenSG}}$, $(\mathsf{ID}_{\mathsf{R}}, \cdot, \cdot) \in \mathcal{L}_{\mathsf{KeyGenR}}$, $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{ID}_{\mathsf{R}}, kw) \neq (\mathsf{ID}_{\mathsf{SG}^*}, \mathsf{ID}_{\mathsf{R}^*}, kw_0^*)$, $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{ID}_{\mathsf{R}}, kw) \neq (\mathsf{ID}_{\mathsf{SG}^*}, \mathsf{ID}_{\mathsf{R}^*}, kw_1^*)$ を満たす $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{ID}_{\mathsf{R}}, kw)$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{ID}_{\mathsf{SG}}, \mathsf{pk}_{\mathsf{SG}}, \mathsf{sk}_{\mathsf{SG}}) \in \mathcal{L}_{\mathsf{KeyGenSG}}$ と $(\mathsf{ID}_{\mathsf{R}}, \mathsf{pk}_{\mathsf{R}}, \mathsf{sk}_{\mathsf{R}}) \in \mathcal{L}_{\mathsf{KeyGenR}}$ を参照し, $\mathsf{C}_{kw} \leftarrow \mathsf{Enc}(\mathsf{pp}, \mathsf{pk}_{\mathsf{SG}},$

© 2026 Information Processing Society of Japan

$\mathsf{pk}_\mathsf{R}, \mathsf{sk}_\mathsf{SG}, kw)$ を実行する．$\mathcal{C}$ は C_kw を返し，記録 $\mathcal{L}_\mathsf{Enc} \leftarrow \mathcal{L}_\mathsf{Enc} \cup \{(\mathsf{ID}_\mathsf{SG}, \mathsf{ID}_\mathsf{R}, kw)\}$ を行う．

トラップドアオラクル: $\mathcal{A}$ は $(\mathsf{ID}_\mathsf{SG}, \cdot, \cdot) \in \mathcal{L}_\mathsf{KeyGenSG}$, $(\mathsf{ID}_\mathsf{SG}, kw) \neq (\mathsf{ID}_{\mathsf{SG}^*}, kw_0^*)$, $(\mathsf{ID}_\mathsf{SG}, kw) \neq (\mathsf{ID}_{\mathsf{SG}^*}, kw_1^*)$ を満たす $(\mathsf{ID}_\mathsf{SG}, kw)$ を $\mathcal{A}$ に送付する．$\mathcal{C}$ は $(\mathsf{ID}_\mathsf{SG}, \mathsf{pk}_\mathsf{SG}, \mathsf{sk}_\mathsf{SG}) \in \mathcal{L}_\mathsf{KeyGenSG}$ を参照，$\mathsf{T}_{kw} \leftarrow \mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk}_{\mathsf{R}^*}, \mathsf{pk}_\mathsf{SG}, \mathsf{sk}_{\mathsf{R}^*}, kw)$ を返し，記録 $\mathcal{L}_\mathsf{Trapdoor} \leftarrow \mathcal{L}_\mathsf{Trapdoor} \cup \{(\mathsf{ID}_\mathsf{SG}, kw)\}$ を行う．

推測: $\mathcal{A}$ は $b' \in \{0,1\}$ を出力する．全ての PPT 攻撃者 $\mathcal{A}$ に対し，利得 $\mathsf{Adv}^{\mathsf{OutsiderTI}}_{\mathsf{PAEKS},\mathcal{A}}(\lambda) = |\Pr[b = b'] - 1/2|$ が λ に対し無視できる場合，PAEKS は Outsider TI を満たすと定義する．

4. 提案一般的構成

提案一般的構成では，$\mathsf{PAEKS} = (\mathsf{SetUp}, \mathsf{KGen}_\mathsf{SG}, \mathsf{KGen}_\mathsf{R}, \mathsf{Enc}, \mathsf{Trapdoor}, \mathsf{Test})$ を，PRP, $\mathsf{NIKE} = (\mathsf{NIKE.SetUp}, \mathsf{NIKE.Gen}, \mathsf{NIKE.SharedKey})$, $\mathsf{PEKS} = (\mathsf{PEKS.SetUp}, \mathsf{PEKS.KGen}, \mathsf{PEKS.Enc}, \mathsf{PEKS.Trapdoor}, \mathsf{PEKS.Test})$ から構成する．

$\mathsf{SetUp}(1^\lambda)$: $\mathsf{pp}_\mathsf{NIKE} \leftarrow \mathsf{NIKE.SetUp}(1^\lambda)$ と $\mathsf{pp}_\mathsf{PEKS} \leftarrow \mathsf{PEKS.SetUp}(1^\lambda)$ を実行し，$\mathsf{pp} = (\mathsf{pp}_\mathsf{NIKE}, \mathsf{pp}_\mathsf{PEKS})$ を出力する．

$\mathsf{KGen}_\mathsf{SG}(\mathsf{pp}, \mathsf{ID}_\mathsf{SG})$: $\mathsf{pp} = (\mathsf{pp}_\mathsf{NIKE}, \mathsf{pp}_\mathsf{PEKS})$, $\mathsf{ID}_\mathsf{SG} = \mathsf{NIKE.ID}_\mathsf{SG}$ とする．$(\mathsf{NIKE.pk}_\mathsf{SG}, \mathsf{NIKE.sk}_\mathsf{SG}) \leftarrow \mathsf{NIKE.Gen}(\mathsf{NIKE.ID}_\mathsf{SG})$ を実行し，$(\mathsf{pk}_\mathsf{SG}, \mathsf{sk}_\mathsf{SG}) = (\mathsf{NIKE.pk}_\mathsf{SG}, \mathsf{NIKE.sk}_\mathsf{SG})$ を出力する．

$\mathsf{KGen}_\mathsf{R}(\mathsf{pp}, \mathsf{ID}_\mathsf{R})$: $\mathsf{pp} = (\mathsf{pp}_\mathsf{NIKE}, \mathsf{pp}_\mathsf{PEKS})$, $\mathsf{ID}_\mathsf{R} = \mathsf{NIKE.ID}_\mathsf{R}$ とする．$(\mathsf{pk}_\mathsf{PEKS}, \mathsf{sk}_\mathsf{PEKS}) \leftarrow \mathsf{PEKS.KGen}(\mathsf{pp}_\mathsf{PEKS})$ と $(\mathsf{NIKE.pk}_\mathsf{R}, \mathsf{NIKE.sk}_\mathsf{R}) \leftarrow \mathsf{NIKE.Gen}(\mathsf{NIKE.ID}_\mathsf{R})$ を実行する．$\mathsf{pk}_\mathsf{R} = (\mathsf{pk}_\mathsf{PEKS}, \mathsf{NIKE.pk}_\mathsf{R})$, $\mathsf{sk}_\mathsf{R} = (\mathsf{sk}_\mathsf{PEKS}, \mathsf{NIKE.sk}_\mathsf{R})$ とし，$(\mathsf{pk}_\mathsf{R}, \mathsf{sk}_\mathsf{R})$ を出力する．

$\mathsf{Enc}(\mathsf{pp}, \mathsf{pk}_\mathsf{SG}, \mathsf{pk}_\mathsf{R}, \mathsf{sk}_\mathsf{SG}, kw)$: $\mathsf{pp} = (\mathsf{pp}_\mathsf{NIKE}, \mathsf{pp}_\mathsf{PEKS})$, $(\mathsf{pk}_\mathsf{SG}, \mathsf{sk}_\mathsf{SG}) = (\mathsf{NIKE.pk}_\mathsf{SG}, \mathsf{NIKE.sk}_\mathsf{SG})$, $\mathsf{pk}_\mathsf{R} = (\mathsf{pk}_\mathsf{PEKS}, \mathsf{NIKE.pk}_\mathsf{R})$ とする．$\mathsf{K}_\mathsf{NIKE} \leftarrow \mathsf{NIKE.SharedKey}(\mathsf{NIKE.ID}_\mathsf{R}, \mathsf{NIKE.pk}_\mathsf{R}, \mathsf{NIKE.ID}_\mathsf{SG}, \mathsf{NIKE.sk}_\mathsf{SG})$, $\mathsf{pr}\text{-}kw \leftarrow F_{\mathsf{K}_\mathsf{NIKE}}(kw)$ と $\mathsf{C}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw} \leftarrow \mathsf{PEKS.Enc}(\mathsf{pp}_\mathsf{PEKS}, \mathsf{pk}_\mathsf{PEKS}, \mathsf{pr}\text{-}kw)$ を実行し，$\mathsf{C}_{kw} = \mathsf{C}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw}$ を出力する．

$\mathsf{Trapdoor}(\mathsf{pp}, \mathsf{pk}_\mathsf{R}, \mathsf{pk}_\mathsf{SG}, \mathsf{sk}_\mathsf{R}, kw)$: $\mathsf{pp} = (\mathsf{pp}_\mathsf{NIKE}, \mathsf{pp}_\mathsf{PEKS})$, $\mathsf{pk}_\mathsf{R} = (\mathsf{pk}_\mathsf{PEKS}, \mathsf{NIKE.pk}_\mathsf{R})$, $\mathsf{sk}_\mathsf{R} = (\mathsf{sk}_\mathsf{PEKS}, \mathsf{NIKE.sk}_\mathsf{R})$, $\mathsf{pk}_\mathsf{SG} = \mathsf{NIKE.pk}_\mathsf{SG}$ とする．$\mathsf{K}_\mathsf{NIKE} \leftarrow \mathsf{NIKE.SharedKey}(\mathsf{NIKE.ID}_\mathsf{SG}, \mathsf{NIKE.pk}_\mathsf{SG}, \mathsf{NIKE.ID}_\mathsf{R}, \mathsf{NIKE.sk}_\mathsf{R})$, $\mathsf{pr}\text{-}kw \leftarrow F_{\mathsf{K}_\mathsf{NIKE}}(kw)$ と $\mathsf{T}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw} \leftarrow \mathsf{PEKS.Trapdoor}(\mathsf{pp}_\mathsf{PEKS}, \mathsf{sk}_\mathsf{PEKS}, \mathsf{pr}\text{-}kw)$ を実行し，$\mathsf{T}_{kw} = \mathsf{T}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw}$ を出力する．

$\mathsf{Test}(\mathsf{C}_{kw}, \mathsf{T}_{kw})$: $\mathsf{C}_{kw} = \mathsf{C}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw}$, $\mathsf{T}_{kw} = \mathsf{T}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw}$ とする．$\mathsf{PEKS.Test}(\mathsf{C}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw}, \mathsf{T}_{\mathsf{PEKS},\mathsf{pr}\text{-}kw})$ を出力する．

NIKE, PRP, PEKS が正当である場合，提案一般的構成は明らかに正当性を満たす．

Theorem1 PEKS が一貫性を，NIKE が HKR-CKS 安全性を，PRP が疑似ランダム性を満たす場合，提案一般的構成は一貫性を満たす．

証明概要: PAEKS の一貫性を PEKS の一貫性に帰着させるために，$\mathcal{A}$ の出力 (i, j, kw, i', j', kw') に対し，変換後キーワード $\mathsf{pr}\text{-}kw$, $\mathsf{pr}\text{-}kw'$ は $\mathsf{pr}\text{-}kw \neq \mathsf{pr}\text{-}kw'$ を満たす必要がある．(i, j, kw, i', j', kw') に対して生成された NIKE 鍵 $\mathsf{K}_{\mathsf{NIKE},i,j}$ と $\mathsf{K}_{\mathsf{NIKE},i',j'}$ は HKR-CKS を用いることで，一様にランダムな鍵に置き換えることができる．続いて $\mathsf{pr}\text{-}kw \leftarrow F_{\mathsf{K}_{\mathsf{NIKE},i,j}}(kw)$ と $\mathsf{pr}\text{-}kw' = F_{K_{\mathsf{NIKE},i',j'}}(kw')$ は真のランダム関数の出力に置き換えることができる．結果，$\mathsf{K}_{\mathsf{NIKE},i,j} \neq \mathsf{K}_{\mathsf{NIKE},i',j'}$ の場合に，$\mathsf{pr}\text{-}kw \neq \mathsf{pr}\text{-}kw'$ であることが示される．

Theorem2 PEKS が IND-CKA 安全性を満たす場合，提案一般的構成は Insider full CI を満たす．

証明概要: PAEKS の insider full CI を PEKS の IND-CKA に帰着させるために，insider full CI を破る攻撃者 $\mathcal{A}$ を用いて，IND-CKA を破るシミュレータ $\mathcal{B}$ を構成する．チャレンジ時の $\mathcal{A}$ の出力 $(\mathsf{ID}_{\mathsf{SG}^*}, kw_0^*, kw_1^*)$ に対し，$\mathcal{B}$ は IND-CKA のチャレンジキーワード $(\mathsf{pr}\text{-}kw_0^*, \mathsf{pr}\text{-}kw_1^*)$ を生成する．ここで，$\mathcal{A}$ が insider full CI を破る場合，$\mathcal{B}$ は明らかに IND-CKA を破る．ただし，$\mathsf{pr}\text{-}kw_0^* = \mathsf{pr}\text{-}kw_1^*$ を満たす場合，$\mathsf{Adv}^{\mathsf{InsiderFullCI}}_{\mathsf{PAEKS},\mathcal{A}}(\lambda) = 0$ となることに留意されたい．しかし，PRP の異なる 2 つの入力に対して，常に 2 つの出力は異なるというという性質上，$kw_0^* \neq kw_1^*$ であれば，この状況は発生しない．

Theorem3 NIKE が HKR-CKS 安全性を，PRP が疑似ランダム性を満たす場合，提案一般的構成は Outsider TI を満たす．

証明概要: $\mathsf{pr}\text{-}kw^*$ に対応するチャレンジキーワード kw_b^* は入力に対し独立であるため，Outsider TI を破る利得は 0 となる．その前に PRP の鍵をランダム鍵に置き換えるが，それは NIKE の HKR-CKS により実現可能である．ただし，$\mathsf{ID}_{\mathsf{SG}^*}$ の公開鍵を HKR-CKS のチャレンジャーによって生成されたものとする必要がある．そのため，何番目に $\mathsf{ID}_{\mathsf{SG}^*}$ の鍵生成が行われたかを推測するフェーズを新たに加える．この推測はセキュリティパラメータに対して多項式回となるため，無視できない確率で成功する．

5. 結論

本論文では，グループ指向 PAEKS を，PRP, NIKE, PEKS からの一般的構成を提案し，従来の PAEKS で送信者数に線形依存していた検索効率を送信者グループ数への線形依存に改善した．提案一般的構成は，ペアリングベースや格子ベースで実現可能である．安全性要件と

© 2026 Information Processing Society of Japan

して, Insider full CI および, 実現不可能性 [27] を考慮し Outsider TI を定義した. Outsider full TI の達成は今後の課題とする. 我々の知る限りでは, PAEKS の検索効率改善を目的としたアプローチの内, 暗号文・トラップドア変換指向 [17, 18, 20, 24, 28] の中で Luo らの手法 [20] に関しては一般的構成が提案されている [21] ものの, サーバー指向 [9, 11, 25] に関しては, 具体的方式の提案のみで, 一般的構成は提案されていない. この一般的構成の提案に関しても今後の課題とする.

謝辞 本研究は JSPS 科研費 JP26K02908 の助成を受けたものです.

参考文献

[1] Michel Abdalla, Mihir Bellare, Dario Catalano, Eike Kiltz, Tadayoshi Kohno, Tanja Lange, John Malone-Lee, Gregory Neven, Pascal Paillier, and Haixia Shi. Searchable encryption revisited: Consistency properties, relation to anonymous IBE, and extensions. *Journal of Cryptology*, 21(3):350–391, 2008.

[2] Daniel Apon, Dana Dachman-Soled, Huijing Gong, and Jonathan Katz. Constant-round group key exchange from the ring-LWE assumption. In *Post-Quantum Cryptography*, pages 189–205, 2019.

[3] Fabrice Benhamouda, Olivier Blazy, Céline Chevalier, David Pointcheval, and Damien Vergnaud. New techniques for sphfs and efficient one-round PAKE protocols. In *CRYPTO, Part I*, pages 449–475, 2013.

[4] Dan Boneh, Giovanni Di Crescenzo, Rafail Ostrovsky, and Giuseppe Persiano. Public key encryption with keyword search. In *EUROCRYPT*, pages 506–522, 2004.

[5] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In *CRYPTO*, pages 213–229, 2001.

[6] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, pages 514–532, 2001.

[7] Xavier Boyen and Brent Waters. Anonymous hierarchical identity-based encryption (without random oracles). In *CRYPTO*, pages 290–307, 2006.

[8] Mike Burmester and Yvo Desmedt. A secure and efficient conference key distribution system (extended abstract). In *EUROCRYPT*, pages 275–286, 1994.

[9] Jiayin Cai, Xingwen Zhao, Dexin Li, Hui Li, and Kai Fan. Efficient and expressive public key authenticated encryption with keyword search in multi-user scenarios. *CoRR*, abs/2503.16828, 2025.

[10] David Cash, Eike Kiltz, and Victor Shoup. The twin Diffie–Hellman problem and applications. *Journal of Cryptology*, 22(4):470–504, 2009.

[11] Leixiao Cheng and Fei Meng. Server-aided public key authenticated searchable encryption with constant ciphertext and constant trapdoor. *IEEE Transactions on Information Forensics and Security*, 19:1388–1400, 2024.

[12] Keita Emura. Generic construction of public-key authenticated encryption with keyword search revisited: Stronger security and efficient construction. In *ACM ASIA Public-Key Cryptography Workshop*, pages 39–49, 2022.

[13] Eduarda S. V. Freire, Dennis Hofheinz, Eike Kiltz, and Kenneth G. Paterson. Non-interactive key exchange. In *Public-Key Cryptography*, pages 254–271, 2013.

[14] Phillip Gajland, Bor de Kock, Miguel Quaresma, Giulio Malavolta, and Peter Schwabe. SWOOSH: efficient lattice-based non-interactive key exchange. In *USENIX Security*, 2024.

[15] Qiong Huang and Hongbo Li. An efficient public-key searchable encryption scheme secure against inside keyword guessing attacks. *Information Sciences*, 403:1–14, 2017.

[16] Jonathan Katz and Vinod Vaikuntanathan. Round-optimal password-based authenticated key exchange. In *Theory of Cryptography*, pages 293–310, 2011.

[17] Hongbo Li, Qiong Huang, Jianye Huang, and Willy Susilo. Public-key authenticated encryption with keyword search supporting constant trapdoor generation and fast search. *IEEE Transactions on Information Forensics and Security*, 18:396–410, 2023.

[18] Hongbo Li, Willy Susilo, Jian Shen, Chen Wang, Leixiao Cheng, and Qiong Huang. Proxy-free public-key authenticated updatable and searchable encryption for cloud storage. *IEEE Transactions on Dependable and Secure Computing*, 23(1):236–249, 2026.

[19] Qinyi Li and Xavier Boyen. Public-key authenticated encryption with keyword search made easy. *IACR Communications in Cryptology*, 1(2):16, 2024.

[20] Fucai Luo, Haiyan Wang, and Xingfu Yan. Re-PAEKS: Public-key authenticated re-encryption with keyword search. *IEEE Transactions on Mobile Computing*, 23(10):10077–10092, 2024.

[21] Daichi Matsushita, Keisuke Hara, and Kyosuke Yamashita. A generic construction of public-key authenticated re-encryption with keyword search. In **信学技報**, *ISEC2026-14*, volume 126, pages 17–24, 2026.

[22] Jae Hong Seo, Tetsutaro Kobayashi, Miyako Ohkubo, and Koutarou Suzuki. Anonymous hierarchical identity-based encryption with constant size ciphertexts. In *Public Key Cryptography*, pages 215–234, 2009.

[23] Wei Wang, Dongli Liu, Zilin Zheng, Peng Xu, and Laurence Tianruo Yang. Identity-based group encryption with keyword search against keyword guessing attack. *IEEE Transactions on Information Forensics and Security*, 19:8023–8036, 2024.

[24] Yongliang Xu, Hang Cheng, Jiguo Li, Ximeng Liu, Xinpeng Zhang, and Meiqing Wang. Lightweight multi-user public-key authenticated encryption with keyword search. *IEEE Transactions on Information Forensics and Security*, 20:3234–3246, 2025.

[25] Wenjie Yang, Shujie Lin, Futai Zhang, and Anjia Yang. Dual-server public key authenticated searchable encryption with constant trapdoor against inside keyword guessing attacks. *IEEE Transactions on Information Forensics and Security*, pages 1–1, 2026 (Early Access).

[26] Takeshi Yoshida and Keita Emura. On the relations between matchmaking public key encryption and public key authenticated encryption with keyword search. In *Cryptology and Network Security*, pages 272–291, 2025.

[27] Takeshi Yoshida and Keita Emura. Comments on "Identity-based group encryption with keyword search against keyword guessing attack". *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 109(7):1314–1315, 2026.

[28] Shengjie Zhang, Jianchang Lai, Liquan Chen, Jinguang Han, and Ge Wu. Keyword-field-free conjunctive searchable encryption for multiuser in EMR system. *IEEE Internet of Things Journal*, 12(15):30850–30861, 2025.

© 2026 Information Processing Society of Japan

付　　録

PRP: PRP が真の乱数関数と識別不可能であることを保証する疑似ランダム性を定義する.

Definition8 (疑似ランダム性)　R は $\{0,1\}^{\ell(\lambda)}$ 上の全ての置換の集合とする. もし全ての PPT 攻撃者 $\mathcal{A}$ に対し, 利得 $\mathsf{Adv}^{\mathsf{Pseudo\text{-}random}}_{\mathsf{PRP},\mathcal{A}}(\lambda) = |\Pr[\mathcal{A}^{F_k(\cdot)}=1] - \Pr[\mathcal{A}^{R(\cdot)}=1]|$ が, λ に対し無視できる場合, PRP は疑似ランダム性を満たすと定義する.

NIKE: Cash, Kiltz, Shoup [10] (CKS) は, NIKE の形式的な安全性として, NIKE 鍵がランダムな鍵と識別不可能であることを保証する安全性である CKS モデルを提案した. さらに, Freire ら [13] は CKS モデルに対し, 階層的な安全性モデルの定義などの拡張を行った. CKS モデルは HKR (Honest Key Registration) -CKS モデルと DKR (Dishonest Key Registration) -CKS モデルに分類され, DKR-CKS の攻撃者は不正に作成した公開鍵を登録可能だが, HKR-CKS の攻撃者に対してはこれを許可しない. Li と Boyen の一般的構成では, 両モデルを定義したうえで, DKR-CKS 安全な NIKE は HKR-CKS 安全を満たすため, 安全性証明では HKR-CKS モデルを用いている. 同様の理由から, 我々も HKR-CKS モデルを用いる. 以下, $\mathcal{C}$ をチャレンジャー, $\mathcal{A}$ を攻撃者とする.

Definition9 (HKR-CKS)　$\mathcal{C}$ は $\mathsf{pp}_{\mathsf{NIKE}} \leftarrow \mathsf{NIKE.SetUp}(1^\lambda)$ を実行し, $\mathsf{pp}_{\mathsf{NIKE}}$ を $\mathcal{A}$ に送付する. $\mathcal{C}$ はオラクルリスト $\mathcal{L} \leftarrow \emptyset$ を定義する. ゲームを通し, $\mathcal{A}$ は以下のオラクルにアクセス可能である.

正規ユーザー ID 登録オラクル:　$\mathcal{A}$ は NIKE.ID を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $(\mathsf{NIKE.pk}, \mathsf{NIKE.sk}) \leftarrow \mathsf{NIKE.Gen}(\mathsf{NIKE.ID})$ を実行し, NIKE.pk を $\mathcal{A}$ に返す. $\mathcal{C}$ は $\mathcal{L} \leftarrow \mathcal{L} \cup \{\mathsf{NIKE.ID}\}$ を実行する.

テストオラクル:　$\mathcal{A}$ は 2 つの識別子 $\mathsf{NIKE.ID}_1, \mathsf{NIKE.ID}_2$ を $\mathcal{C}$ に送付する. $\mathsf{NIKE.ID}_1 = \mathsf{NIKE.ID}_2$, $\mathsf{NIKE.ID}_1 \notin \mathcal{L}$, $\mathsf{NIKE.ID}_2 \notin \mathcal{L}$ のいずれかの場合, $\mathcal{C}$ は $\perp$ を出力する. $\mathcal{C}$ はチャレンジビット $b \xleftarrow{\$} \{0,1\}$ を選択し, $\mathsf{K}_{\mathsf{NIKE},0} \leftarrow \mathsf{NIKE.SharedKey}(\mathsf{NIKE.ID}_1, \mathsf{NIKE.pk}_1, \mathsf{ID}_2, \mathsf{NIKE.NIKE.sk}_2)$ を実行し, さらに, ランダムな鍵 $\mathsf{K}_{\mathsf{NIKE},1} \leftarrow \{0,1\}^\lambda$ を生成する. $\mathcal{C}$ は b に対応する鍵 $\mathsf{K}_{\mathsf{NIKE},b}$ を $\mathcal{C}$ に返す.

推測:　$\mathcal{A}$ は, $b' \in \{0,1\}$ を出力する. 全ての PPT 攻撃者 $\mathcal{A}$ に対し, 利得 $\mathsf{Adv}^{\mathsf{HKR\text{-}CKS}}_{\mathsf{NIKE},\mathcal{A}}(\lambda) = |\Pr[b=b'] - 1/2|$ が λ に対し無視できる場合, NIKE は HKR-CKS 安全性を満たすと定義する.

PEKS: PEKS の機能要件として, 正当性を定義する. これは正規のアルゴリズムに従って生成された暗号文 $\mathsf{C}_{\mathsf{PEKS},kw}$ とトラップドア $\mathsf{T}_{\mathsf{PEKS},kw'}$ に対し, $kw = kw'$ の場合, テストアルゴリズムが常に 1 を返すことを保証するものである.

Definition10 (正当性)　全ての $\lambda \in \mathbb{N}, \mathsf{pp}_{\mathsf{PEKS}} \leftarrow \mathsf{SetUp}(1^\lambda), (\mathsf{pk}_{\mathsf{PEKS}}, \mathsf{sk}_{\mathsf{PEKS}}) \leftarrow \mathsf{PEKS.KGen}(\mathsf{pp}_{\mathsf{PEKS}}), kw \in \mathcal{KW}$ に対し, $1 \leftarrow \mathsf{Test}(\mathsf{PEKS.Enc}(\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{pk}_{\mathsf{PEKS}}, kw), \mathsf{PEKS.Trapdoor}(\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{sk}_{\mathsf{PEKS}}, kw))$ が常に出力される場合, PEKS は正当性を満たすと定義する.

続いて, 機能要件として, 一貫性を定義する. これは暗号化したキーワード kw とトラップドアに紐づくキーワード kw' に対し, $kw \neq kw'$ の場合, テストアルゴリズムが常に 0 を返すことを保証するものである.

Definition11 (一貫性)　$\mathcal{C}$ は $\mathsf{pp}_{\mathsf{PEKS}} \leftarrow \mathsf{PEKS.SetUp}(1^\lambda)$, $(\mathsf{pk}_{\mathsf{PEKS}}, \mathsf{sk}_{\mathsf{PEKS}}) \leftarrow \mathsf{PEKS.KGen}(\mathsf{pp}_{\mathsf{PEKS}})$ を実行し, $\mathsf{pk}_{\mathsf{PEKS}}$ を $\mathcal{A}$ に送付する. $\mathcal{A}$ は $(kw, kw') \in \mathcal{KW}^2$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $\mathsf{C}_{\mathsf{PEKS},kw} \leftarrow \mathsf{PEKS.Enc}(\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{pk}_{\mathsf{PEKS}}, kw)$ と $\mathsf{T}_{\mathsf{PEKS},kw'} \leftarrow \mathsf{PEKS.Trapdoor}(\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{sk}_{\mathsf{PEKS}}, kw')$ を実行し, これらに対し $kw \neq kw'$ かつ $1 \leftarrow \mathsf{PEKS.Test}(\mathsf{C}_{\mathsf{PEKS},kw}, \mathsf{T}_{\mathsf{PEKS},kw'})$ を満たす場合に $\mathcal{A}$ は勝利すると定義する. 全ての PPT 攻撃者 $\mathcal{A}$ に対し, 利得 $\mathsf{Adv}^{\mathsf{consist}}_{\mathsf{PEKS},\mathcal{A}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$ が λ に対し無視できる場合, PEKS は一貫性を満たすと定義する.

PEKS の安全性要件として, 選択キーワード攻撃に対する識別不可能性 (IND-CKA) を定義する. これは, 暗号文からキーワードに関する情報が漏洩しないことを保証するものである.

Definition12 (IND-CKA)　$\mathcal{C}$ は $\mathsf{pp}_{\mathsf{PEKS}} \leftarrow \mathsf{PEKS.SetUp}(1^\lambda), (\mathsf{pk}_{\mathsf{PEKS}}, \mathsf{sk}_{\mathsf{PEKS}}) \leftarrow \mathsf{PEKS.KGen}(\mathsf{pp}_{\mathsf{PEKS}})$ を実行し, $\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{pk}_{\mathsf{PEKS}}$ を $\mathcal{A}$ に送付する.

Phase I:　$\mathcal{A}$ は以下のオラクルを利用可能.

トラップドアオラクル:　$\mathcal{A}$ は kw を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $\mathsf{T}_{\mathsf{PEKS},kw} \leftarrow \mathsf{PEKS.Trapdoor}(\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{sk}_{\mathsf{PEKS}}, kw)$ を実行し, $\mathsf{T}_{\mathsf{PEKS},kw}$ を $\mathcal{A}$ に返す.

チャレンジ:　$\mathcal{A}$ はチャレンジキーワード (kw_0^*, kw_1^*) を宣言する. ただし, (kw_0^*, kw_1^*) は Phase I でトラップドアオラクルに送付していないものとする. $\mathcal{C}$ は $b \xleftarrow{\$} \{0,1\}$ を選択し, チャレンジ暗号文 $\mathsf{C}_{\mathsf{PEKS},kw_b^*} \leftarrow \mathsf{PEKS.Enc}(\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{pk}_{\mathsf{PEKS}}, kw_b^*)$ を $\mathcal{A}$ に送付する.

Phase II:　$\mathcal{A}$ は以下のオラクルを利用可能.

トラップドアオラクル:　$\mathcal{A}$ は $kw \notin \{kw_0^*, kw_1^*\}$ を $\mathcal{C}$ に送付する. $\mathcal{C}$ は $\mathsf{T}_{\mathsf{PEKS},kw} \leftarrow \mathsf{PEKS.Trapdoor}(\mathsf{pp}_{\mathsf{PEKS}}, \mathsf{sk}_{\mathsf{PEKS}}, kw)$ を実行し, $\mathsf{T}_{\mathsf{PEKS},kw}$ を $\mathcal{A}$ に返す.

推測:　$\mathcal{A}$ は $b' \in \{0,1\}$ を出力する. 全ての PPT 攻撃者 $\mathcal{A}$ に対し, 利得 $\mathsf{Adv}^{\mathsf{IND\text{-}CKA}}_{\mathsf{PEKS},\mathcal{A}}(\lambda) = |\Pr[b=b'] - 1/2|$ が λ に対し無視できる場合, PEKS は IND-CKA 安全性を満たすと定義する.

© 2026 Information Processing Society of Japan